

Graph Data Augmentation Using Generative Adversarial Networks for Imbalanced Anti-Money Laundering Datasets

Shahram Ghahremani*, Uyen Trang Nguyen†

Department of Electrical Engineering & Computer Science, York University, Toronto, Canada

Email: *shg@yorku.ca, †unguyen@yorku.ca

Abstract—One of the challenges in developing machine learning models for money laundering detection is the severe class imbalance in anti-money laundering (AML) datasets. To mitigate class imbalance, oversampling methods such as SMOTE and ADASYN are commonly used to create additional samples for the minority class. However, these methods operate on flat feature representations and fail to capture graph-based structures that are characteristic of money laundering behaviors. In this paper, we propose a model based on Conditional Generative Adversarial Networks (CGAN) to generate synthetic money laundering graph patterns that preserve both transaction attributes and graph connectivity of underlying AML data. Experimental results on multiple datasets show that the proposed CGAN model generates synthetic data that enable money laundering detection models to achieve significantly higher predictive performance in terms of recall and false alert rates, compared to SMOTE and ADASYN.

Index Terms—financial crimes, money laundering detection, synthetic data generation, graph data, transaction graphs, conditional generative adversarial networks, graph neural networks, class imbalance, oversampling

I. INTRODUCTION

Money laundering (ML) refers to the process of disguising the origins of illicit funds by moving them through layers of financial transactions and entities, ultimately making the funds appear legitimate. It is a key enabler of criminal enterprises, including tax evasion, corruption, and terrorism. Regulatory frameworks require financial institutions to implement anti-money laundering (AML) controls, such as know-your-customer (KYC) procedures, risk assessment, and transaction monitoring. Failure to comply with these regulations has led to significant financial penalties and reputational harm, with global fines reaching billions of dollars (2).

Among AML measures, transaction monitoring plays a central role by flagging potentially suspicious activity for further investigation. Traditional AML systems rely on rules-based algorithms that apply fixed thresholds or typologies to detect suspicious patterns (3). Although effective in maintaining regulatory compliance, these systems tend to produce high false alert rates—often exceeding 90%—which incur significant costs of manual review and investigation (4). (The false alert rate is the number of false positives divided by the total number of alerts generated.) Machine learning has proven to be effective at reducing false positives while maintaining the same detection rate (5; 6). However, a core limitation remains: the extreme class imbalance in AML datasets, where illicit transactions

account for less than 0.1% of all transactions (7; 8; 9). Models trained on imbalanced data often perform very poorly on the minority class (money laundering samples in this case), leading to very low detection rates.

To address this imbalance, data-level methods such as undersampling and oversampling (e.g., random oversampling, SMOTE (10), ADASYN (11)) have been widely used (12; 13), alongside algorithm-level approaches like cost-sensitive learning and anomaly detection (14; 15). While these methods offer some improvements, they are often constrained by design. Undersampling may lead to information loss. Cost-sensitive learning alone is not effective due to the extreme imbalance of AML data, as will be shown in Section V-B. We thus study the effectiveness of data augmentation (oversampling) of the minority class in this paper. However, oversampling with flat feature vectors by SMOTE or ADASYN fails to preserve the complex interactions found in financial transactions, as such methods synthesize isolated samples based on numeric interpolation. Consequently, they do not capture the structural dependencies between entities involved in coordinated laundering schemes.

Money laundering is typically carried out through coordinated behaviors that span multiple transactions and accounts. These behaviors form structured patterns across transaction networks, involving repeated transfers, loops, and intermediary entities to conceal the flow of funds. Capturing these patterns requires machine learning models that understand both the features of individual transactions and the relationships between them. Oversampling approaches that ignore such structural context are limited in their ability to support graph-based classifiers, which rely on topological information to detect suspicious activity.

In this paper, we propose a data augmentation method based on *Conditional Generative Adversarial Networks* (CGANs) (16) for synthetic AML data generation. CGANs extend standard GANs by incorporating auxiliary information, enabling the generation of samples that reflect specific class or pattern-level characteristics. In our framework, CGAN learns to generate realistic transaction networks that resemble laundering patterns, preserving both transaction-level features and inter-entity relationships. Unlike traditional oversampling techniques, CGAN creates synthetic transaction patterns—complete with structural integrity and attribute consistency—that are suitable for training graph-based AML detection models.

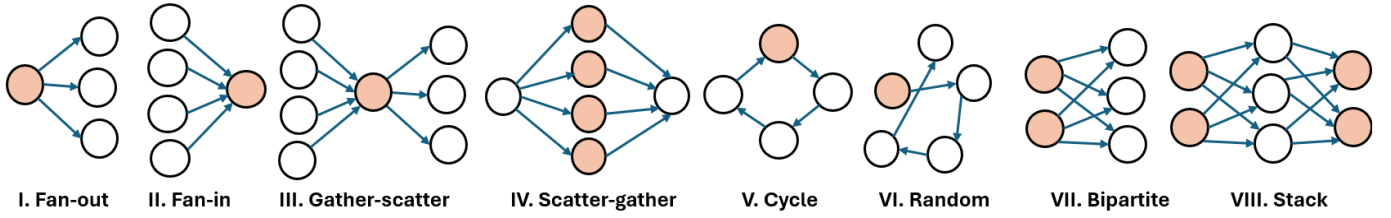


Fig. 1: Money laundering patterns (1)

The main contributions of this paper are as follows:

- We propose a CGAN-based oversampling method to generate structured, class-conditional synthetic transaction graphs for AML datasets.
- Our approach captures both transaction attributes and graph connectivity, producing high-fidelity samples that resemble known money laundering patterns.
- We evaluate the proposed CGAN-based data augmentation method on four benchmark datasets from the AMLWorld data suite (1) using several graph neural network (GNN) classifiers to assess downstream application performance, money laundering detection in this case.
- Experimental results show that CGAN-generated data reduce false alert rates while maintaining high detection rates (recall), outperforming SMOTE and ADASYN in terms of both statistical similarity and detection performance.

To our knowledge, our work is the first that generates synthetic money laundering data samples at the pattern level, and evaluates the effectiveness of the synthetic data in the downstream application of money laundering detection.

The remainder of this paper is organized as follows. Section II reviews related work, including key money laundering patterns and existing methods for handling class imbalance in AML. Section III presents the proposed CGAN model for generating synthetic money laundering data. Section IV describes the experimental setup, including datasets, baselines, and evaluation metrics. Section V reports experimental results, focusing on evaluations of data similarity and money laundering detection performance. Section VI concludes the paper and outlines future directions.

II. RELATED WORK

A. Money Laundering Patterns

Unlike fraud detection, where a single unusual transaction—such as a location mismatch or a large online purchase—can indicate fraud, money laundering involves multiple participants and transactions across different stages (placement, layering, and integration (17)) to obscure the money’s origin.

To achieve this, money launderers use various techniques. Suzumura and Kanezashi (18) identified eight key money laundering patterns (Fig. 1), which are also simulated in datasets such as AMLSim (19) and AMLWorld (1).

One of these patterns is the **fan-out** structure, illustrated in Figure 1.I. In this pattern, a particular node (an account) distributes funds to at least $N \geq 2$ distinct recipients through

outgoing edges. Conversely, the **fan-in** pattern, depicted in Figure 1.II, occurs when multiple nodes transfer funds to a single node via incoming edges. The **gather-scatter** pattern, shown in Figure 1.III, is a combination of fan-in and fan-out, where a single node both receives funds from multiple sources and subsequently disperses them to multiple destinations.

Another complex pattern, the **scatter-gather** structure, consists of two interconnected components: a fan-out from one node n and a fan-in into another node m , with both connected through the same intermediary nodes (Figure 1.IV). The **simple cycle** pattern, represented in Figure 1.V, consists of transactions forming a closed loop, where funds move through a sequence of vertices before returning to the starting point.

The **random pattern**, illustrated in Figure 1.VI, resembles the cycle pattern but lacks a direct return to the originating account. Instead, the funds move unpredictably between accounts, often controlled through shell companies or other means. The **bipartite** pattern, shown in Figure 1.VII, contains transactions between two distinct groups of accounts, where funds flow from a set of input accounts to a set of output accounts. Finally, the **stack** pattern, visualized in Figure 1.VIII, expands on the bipartite model by introducing an extra bipartite layer, further obfuscating the transaction trail.

B. Approaches to Handling Class Imbalance

The problem of extreme class imbalance is a well-known challenge in machine learning, especially in high-stakes domains such as fraud detection, medical diagnosis, and anti-money laundering (AML). While a broad body of literature surveys class imbalance strategies across different application areas (20; 21; 22; 23), AML presents unique challenges due to the sparsity and complexity of illicit transaction patterns (24; 25). The lack of minority class samples means that an ML detection model does not learn to recognize or correctly classify ML cases, which may lead to regulatory violation and monetary fines.

Existing solutions to the class imbalance problem can be broadly categorized into data-level and algorithm-level approaches (26; 27). Data-level methods focus on rebalancing the training set via resampling methods, while algorithm-level approaches modify the learning process to give higher weight to rare class instances.

Table I provides a summary of techniques used to mitigate class imbalance in existing money laundering detection models.

TABLE I: Data-level and algorithm-level methods in existing money laundering detection models

Model Name	Data-Level		Algorithm-Level		Methods Used
	Undersampling	Oversampling	Weighted Loss	One-Class Learning	
Pambudi et al. (2023) (12)	✓				Random undersampling
Jullum et al. (2020) (5)	✓				Random undersampling
Girish et al. (2024) (28)		✓			SMOTE
Sintayehu et al. (2023) (29)		✓			SMOTE
Caglayan et al. (2022) (30)		✓			SMOTE, ADASYN
Assumpcao et al. (2022) (31)		✓			GraphSMOTE
Zhang et al. (2019) (13)	✓	✓			Random undersampling & oversampling
Bakhshinejad et al. (2024) (32)	✓	✓			SMOTE and near-miss undersampling
Altman et al. (2023) (1)			✓		Weighted binary cross-entropy loss
Schmidt et al. (2025) (14)			✓		Weighted binary cross-entropy loss
Luo et al. (2022) (33)			✓		Focal loss
Tertychnyi et al. (2022) (15)				✓	Isolation forests
Baltoiu et al. (2019) (34)				✓	One-class SVM
El Kilany et al. (2024) (35)				✓	One-class SVM

1) **Data-Level Approaches:** Data-level techniques adjust the class distribution before model training. Undersampling methods, such as random undersampling (RUS) (12; 5), reduce the size of the majority class to rebalance the dataset. More targeted approaches, like near-miss undersampling (32), preserve majority samples that are close to the minority class boundary. However, these methods often discard informative data and risk losing valuable context.

Oversampling methods generate synthetic samples to augment the minority class. SMOTE (28; 29) interpolates between existing minority samples to generate new instances, while ADASYN (30) focuses on under-represented regions of the minority class. Though effective at improving the performance of the minority class, both methods operate on flat feature spaces and fail to account for inter-transaction dependencies. Some graph-based extensions, such as GraphSMOTE (31), aim to address this limitation, though they introduce additional computational complexity.

Hybrid resampling techniques combine both undersampling and oversampling to balance the trade-off between redundancy and information loss. For example, Zhang et al. (13) use a blend of RUS and random oversampling (ROS), while Bakhshinejad et al. (32) integrate SMOTE with near-miss sampling for improved AML classification performance.

2) **Algorithm-Level Approaches:** Algorithm-level approaches focus on modifying the learning objective rather than altering the data distribution. Cost-sensitive learning assigns higher penalties for misclassifying minority-class instances. This has been implemented through weighted loss functions in several AML studies (1). Altman et al. applied weighted binary cross-entropy loss to mitigate imbalance in both GBT and GNN models, while Schmidt et al. designed GAMLNet using a similar cost-weighted strategy.

Other works employ loss functions that dynamically adapt to data difficulty. For example, Luo et al. (33) apply focal loss to

focus on harder-to-classify minority samples. However, these approaches often require careful hyperparameter tuning and can still underperform under extreme imbalance.

A different class of algorithm-level techniques reframes the task as anomaly detection, thereby avoiding class labeling altogether. Tertychnyi et al. (15) used Isolation Forest to detect behavioral outliers in transactional communities. Similarly, El Kilany et al. (35) and Baltoiu et al. (34) used one-class SVMs to identify anomalous customer profiles. While effective in certain settings, anomaly detection models often suffer from high false positive rates and lack interpretability.

C. Limitations of the Above Methods and Our Proposed Approach

Despite the progress made by these techniques, several challenges remain unresolved. Undersampling discards data and reduces model robustness. Oversampling methods like SMOTE and ADASYN treat transactions as independent data points, ignoring the relational and sequential structures intrinsic to money laundering schemes. Cost-sensitive learning may fail under extreme imbalance, and anomaly detection approaches often yield high false alert rates without capturing known laundering typologies.

To overcome these limitations, we propose a structured data generation framework based on *Conditional Generative Adversarial Networks* (CGANs). Unlike traditional oversampling, CGANs generate realistic subgraphs conditioned on laundering pattern types or contextual features, capturing both the transactional and structural dependencies essential for AML detection. This approach provides a principled way to augment minority-class samples with high-fidelity, pattern-aware data and enables the training of graph-based classifiers on enriched, semantically meaningful datasets.

To the best of our knowledge, this is the first application of CGANs to generate pattern-level synthetic AML data for transaction monitoring. Our method addresses the shortcomings

of previous oversampling strategies by synthesizing full illicit behavior structures rather than isolated data points, thereby aligning with how real-world laundering operations manifest in transaction networks.

D. Limitations of the Above Methods and Our Proposed Approach

Despite the progress made by these techniques, several challenges remain unresolved. Undersampling discards data and reduces model robustness. Oversampling methods like SMOTE and ADASYN treat transactions as independent data points, ignoring the relational and sequential structures intrinsic to money laundering schemes. Cost-sensitive learning may fail under extreme imbalance, and anomaly detection approaches often yield high false alert rates without capturing known laundering typologies.

To overcome these limitations, we propose a structured data generation framework based on *Conditional Generative Adversarial Networks* (CGANs). Unlike traditional oversampling, CGANs generate realistic subgraphs conditioned on laundering pattern types or contextual features, capturing both the transactional and structural dependencies essential for AML detection. This approach provides a principled way to augment minority-class samples with high-fidelity, pattern-aware data and enables the training of graph-based classifiers on enriched, semantically meaningful datasets.

To the best of our knowledge, this is the first application of CGANs to generate pattern-level synthetic AML data for transaction monitoring. Our method addresses the shortcomings of previous oversampling strategies by synthesizing full illicit behavior structures rather than isolated data points, thereby aligning with how real-world laundering operations manifest in transaction networks.

III. THE PROPOSED MODEL FOR SYNTHETIC GRAPH DATA GENERATION

We propose a CGAN model for generating synthetic money laundering patterns to address the class imbalance of anti-money laundering (AML) datasets. The proposed model synthesizes graph-structured transaction data conditioned on known money laundering patterns (Fig. 1), allowing minority-class samples to be augmented in a structure-aware and semantically meaningful manner. An overview of the model pipeline is shown in Figure 2, which illustrates how transaction records are transformed into graph representations and used for pattern-conditioned generation.

To prepare the input for training, we extracted money laundering patterns (e.g., fan-in, fan-out, cycle, etc.) as described in Section II-A. Each extracted instance is a graph encoded as follows:

- an *adjacency matrix* $A \in \{0, 1\}^{n \times n}$ representing the graph;
- a *feature matrix* $F \in \mathbb{R}^{n \times d}$ containing attributes such as timestamp, transaction amount, currency type, and transfer method;

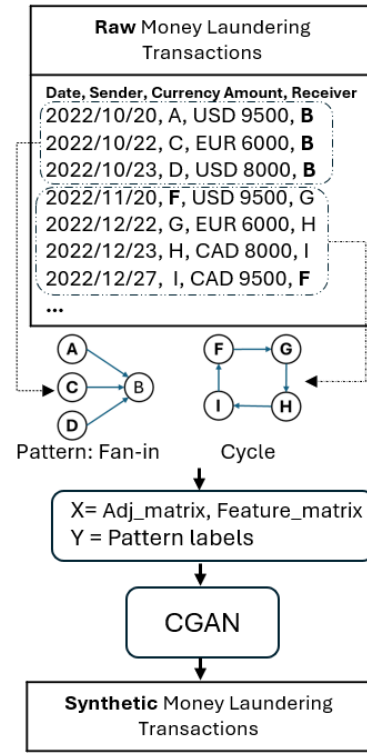


Fig. 2: Overview of the synthetic graph data generation pipeline

- a *label* Y indicating the pattern type of the graph (e.g., fan-in, fan-out, gather-scatter, scatter-gather, etc.).

The pattern graphs have different sizes, resulting in matrices of different sizes. Therefore, all input matrices are padded to a uniform size of 45×45 to standardize dimensions across samples. The input tuple (A, F) is denoted as X , and (X, Y) pairs are used to train the CGAN.

A. Conditional Generative Adversarial Network (CGAN)

The CGAN architecture extends the standard Generative Adversarial Network (GAN) by incorporating conditioning variables (16), enabling label-aware generation. It comprises two neural components:

- The **generator** $G(z, Y)$, which maps a noise vector $z \sim \mathcal{N}(0, I)$ and pattern label Y to a synthetic graph $\hat{X} = (\hat{A}, \hat{F})$;
- The **discriminator** $D(X, Y)$, which attempts to distinguish between real and generated input-label pairs.

The two networks are trained in an adversarial manner: D learns to identify synthetic samples, while G learns to produce outputs that D cannot distinguish from real examples. The training objective is given by the following minimax game:

$$\min_G \max_D \mathcal{L}_{CGAN} = \mathbb{E}_{X, Y} [\log D(X, Y)] + \mathbb{E}_{z, Y} [\log (1 - D(G(z, Y), Y))].$$

The conditioning label Y plays a central role, guiding both G and D to align the structure and semantics of generated samples with specific pattern types. Unlike variational autoencoders (VAEs), which use reconstruction and Kullback–Leibler (KL)

divergence losses to enforce latent distribution regularization, CGAN relies entirely on adversarial loss. This allows the generator to learn sharper, more complex distributions without explicit probabilistic constraints, albeit at the cost of more challenging training dynamics.

The discriminator and generator are implemented using fully connected layers that jointly process the adjacency and feature matrices. During inference, the generator produces label-specific samples $\hat{X}$ that are used to augment the minority class in the original dataset.

B. Graph-Based Representation and Augmentation

Following generation, the synthetic samples are post-processed into graph objects using the `NetworkX` library. Nodes represent accounts, and directed edges represent transactions between accounts. These synthetic graph structures are combined with the original graph data to obtain more balanced datasets, which are subsequently used to train graph neural network (GNN) classifiers to detect money laundering.

IV. EXPERIMENT SETUP

This section discusses the datasets, experimental setup, and metrics used to evaluate the performance of the proposed CGAN-based data augmentation method.

A. Dataset

Real-world financial transaction data is rarely available due to privacy and proprietary restrictions. Furthermore, even in accessible datasets, ground truth labels for illicit transactions are often unreliable. This is due to the inherently covert nature of money laundering activities, which leads to extremely low positive class densities (e.g., 0.1%–1%) and ambiguous transactional patterns that may resemble legitimate behavior. Consequently, labeling is often derived from heuristics or sparse investigative confirmation, introducing bias and label noise. To address these challenges in a controlled setting, we use the publicly available AMLWorld dataset developed by IBM Research (1; 9). AMLWorld simulates a financial network consisting of individual and corporate entities, and models the full money laundering lifecycle.

AMLWorld provides two variants categorized by laundering prevalence: **HI** (High Illicit Ratio) and **LI** (Low Illicit Ratio). Each variant is further divided into *small*, *medium*, and *large* datasets, enabling evaluation across varying transaction volumes and illicit transaction densities. To mitigate extreme class imbalance, we apply our CGAN model to generate synthetic laundering patterns, which are then appended to the original data. Running the GNN models on the large dataset of AMLWorld requires substantial computational resources and excessive runtime; therefore, we present results only for the small and medium datasets of AMLWorld. Table II summarizes statistics before and after data augmentation. Table III shows the number of instances (graphs) of each illicit pattern type before and after data augmentation for each dataset.

We formulate the task as an edge classification problem: nodes correspond to accounts and edges represent transactions

between accounts. We employ a temporally ordered 60/20/20 split for training, validation, and testing, preserving chronological consistency and preventing data leakage. This setting emulates realistic detection pipelines where future transactions are predicted from past behavior.

B. Baseline Synthetic Data Generation Methods

To assess the effectiveness of our CGAN-based approach for generating structured laundering patterns, we compare it against two commonly used synthetic data generation techniques: SMOTE (10) and ADASYN (11), the most widely used techniques for addressing class imbalance in fraud detection and anti-money laundering applications (10; 11; 28; 29; 30). SMOTE and ADASYN perform oversampling by interpolating between minority-class samples in feature space, without regard for transactional structure. In contrast, CGAN learns to generate samples conditioned on class labels, making it suitable for producing structured graph-based data.

C. Evaluation Methods

We evaluate the effectiveness of our CGAN-based model using two criteria: (1) statistical similarity between synthesized and original laundering patterns, and (2) the effectiveness of synthetic data on the performance of the downstream task of money laundering detection.

1) *Statistical Similarity*: To evaluate statistical similarity, we use the *Wasserstein distance* (36), a well-established metric for comparing probability distributions. The Wasserstein distance quantifies the effort required to transform one distribution into another, making it particularly suitable for assessing the similarity between real and synthetic data in terms of their underlying statistical structure. Unlike simpler metrics that only compare marginal distributions (e.g., the Kolmogorov-Smirnov test (37), which evaluates the similarity of one-dimensional distributions for individual features), the Wasserstein distance considers the joint distribution of features and the geometry of the data space. This approach allows for a nuanced evaluation of how closely the synthetic data replicates the true statistical properties of the original data, particularly in capturing complex variations in local and global density. A smaller Wasserstein distance indicates a higher quality of synthetic data, reflecting greater statistical similarity to real data.

2) *Effectiveness on Downstream Classification*: To evaluate the utility of CGAN-generated data in improving money laundering detection, we adopt the AMLWorld experimental protocol presented in (1). Specifically, we train three GNN-based classifiers used in (1) on the original and CGAN-augmented datasets:

- **GIN** (Graph Isomorphism Network) (38), which uses sum aggregation to capture graph topology;
- **GIN+EU** (39), which extends GIN with edge update mechanisms to better model transactional dependencies;
- **PNA** (Principal Neighbourhood Aggregation) (40), which combines multiple aggregation statistics (e.g., mean, max, standard deviation) with degree-scalers to enhance generalization.

TABLE II: Sizes of the original and augmented datasets. HI: high illicit transaction ratio; LI: low illicit transaction ratio; M: million; K: thousand.

Statistics	Small dataset				Medium dataset			
	HI		LI		HI		LI	
	Original	Augmented	Original	Augmented	Original	Augmented	Original	Augmented
Total Number of Transactions	5.1M	5.2M	6.9M	7.0M	31.9M	32.5M	31.3M	31.5M
Number of Laundering Transactions	5.2K	94.8K	3.6K	65.3K	35.2K	644.9K	16.0K	293.6K
Minority-to-Majority Sample Ratio	1:981	1:54	1:1942	1:107	1:905	1:50	1:1948	1:107

TABLE III: Numbers of money laundering samples (graphs) in the original and augmented datasets

Pattern	Small dataset				Medium dataset			
	HI		LI		HI		LI	
	Original	Augmented	Original	Augmented	Original	Augmented	Original	Augmented
Fan-Out	48	874	19	346	345	6279	69	1256
Fan-In	40	728	12	218	355	6451	47	855
Random	41	746	15	273	331	6024	49	892
Bipartite	49	892	16	291	369	6716	72	1310
Stack	43	783	18	328	336	6115	67	1219
Cycle	54	983	12	218	367	6679	46	837
Scatter-Gather	44	801	13	237	331	6024	63	1147
Gather-Scatter	51	928	12	218	322	5850	43	783

To mitigate class imbalance in the original datasets, we applied a class-weighted loss function during training. Additional model details and training configurations follow those in (1).

Classification Metrics: Accuracy is not suitable for AML tasks due to the extreme class imbalance. For example, with a 1:1000 positive-to-negative sample ratio, a model that always predicts the negative class can achieve 99.9% accuracy. F1-score is not suitable either because it favors a balance of recall and precision. However, in AML, the cost of missed money laundering cases (low recall) is much higher than the cost of false positives (low precision); thus recall (or detection rate) is given priority over precision.

In practice, recall (detection rate or true positive rate) is the primary metric of interest, as missing money laundering cases can lead to monetary penalties and reputational harm. However, maximizing recall often leads to high false alert rates (FAR). Typical rules-based systems operate at FARs exceeding 95%, i.e., precision as low as 5%. Investigating these false positives incurs substantial operational costs.

Accordingly, we report both *recall* and *FAR* to evaluate classifier performance. The goal is to achieve high recall (e.g., 97%) while reducing the FAR as much as possible, thereby improving both detection rate and operational efficiency.

D. Hyperparameter Tuning of the CGAN

Table IV summarizes the hyperparameter search spaces and selected values for the CGAN used to generate synthetic money laundering data. The primary hyperparameters include the noise vector dimension, learning rate, number of hidden layers in the generator and discriminator, and the number of hidden units in each layer. Grid search was used to identify the optimal

configuration for each hyperparameter based on validation performance, balancing fidelity and training stability. For each configuration, five CGAN models were trained with different random seeds, and the average and standard deviation of their output are reported to ensure statistical robustness.

The final architecture uses a 32-dimensional noise vector, a learning rate of 0.001, and two-layer architectures for both generator and discriminator, with hidden unit sizes tuned separately per layer. These settings consistently yielded the best trade-off between generator convergence and downstream detection performance.

TABLE IV: Hyperparameter ranges and selected values used in the CGAN model

Hyperparameter	Range	Selected Value
Noise vector dimension (input to G)	(8, 64)	32
Learning rate	(0.0001, 0.01)	0.001
Number of generator layers	(1, 4)	2
Number of discriminator layers	(1, 4)	2
Generator units – layer 1	(64, 512)	128
Generator units – layer 2	(64, 512)	256
Discriminator units – layer 1	(64, 512)	256
Discriminator units – layer 2	(64, 512)	128
Dropout rate (discriminator only)	(0, 0.5)	0.3

V. EXPERIMENTAL RESULTS

We evaluate the quality of synthetic data generated by SMOTE, ADASYN, and our CGAN-based model. Evaluation includes both data similarity metrics and downstream money laundering detection performance.

TABLE V: Wasserstein distance scores of the three data augmentation methods for the four datasets

Method	Small		Medium	
	HI	LI	HI	LI
SMOTE	0.0626 $\pm$ 0.0071	0.0619 $\pm$ 0.0099	0.0620 $\pm$ 0.0026	0.0637 $\pm$ 0.0088
ADASYN	0.1053 $\pm$ 0.0696	0.1065 $\pm$ 0.0764	0.1063 $\pm$ 0.0230	0.1068 $\pm$ 0.0643
CGAN	0.0726 $\pm$ 0.0030	0.0728 $\pm$ 0.0098	0.0725 $\pm$ 0.0050	0.0725 $\pm$ 0.0061

A. Data Similarity Evaluation

We evaluate the statistical similarity between the synthetic and original transaction data using the Wasserstein distance metric on the datasets listed in Table II. Table V reports the average Wasserstein scores for SMOTE, ADASYN, and CGAN across different dataset sizes and class distributions.

SMOTE-synthesized data achieve the lowest Wasserstein distances (between 0.0619 and 0.0637), indicating high statistical similarity to the original data. This reflects its strength in preserving local feature distributions through interpolation between existing samples. ADASYN data show the highest distances, while CGAN occupies a middle ground, with moderate Wasserstein distances across all datasets.

Most importantly, higher Wasserstein distances do not imply lower utility. As will be discussed in Section IV-C2, CGAN-augmented datasets consistently outperform SMOTE data in terms of the performance of the money laundering detection models. This underscores a key insight: close statistical alignment with real data is not always indicative of higher detection performance, particularly in AML data where structural patterns are as important as feature distributions.

B. Evaluation of Synthetic Data on Money Laundering Detection

We evaluate the effectiveness of CGAN-generated data on the downstream task of money laundering detection using three GNN architectures: GIN (38), GIN+EU (39), and PNA (40) as in (1). The GNN models are trained on the original AMLWorld datasets and three more balanced datasets augmented by CGAN, SMOTE, and ADASYN. The test sets retain the natural class distributions.

The evaluation metrics are recall (detection rate) and false alert rate ($\text{FAR} = 1 - \text{precision}$), consistent with AML priorities that emphasize recall to minimize missed illicit activity. We report results for two recall targets of approximately 90% and 97%. We adjusted the intersection-over-union (IoU) threshold to achieve the target recall for each GNN model. At the same recall, the model with the lower FAR (and thus lower operational costs) is considered the higher performer. We also conducted experiments with the standard threshold of 0.5 commonly used in many machine learning tasks. (However, the standard threshold is not suitable for money laundering detection, which prioritizes detection rates over false positives. These results are included for comparison purposes only.)

The results from the HI-Medium dataset (Table VI) show:

- All GNN models trained on augmented data achieve higher performance (lower FARs at the same target recall) compared to those trained on the original data (with natural distributions).
- GNN models trained on CGAN-augmented data consistently outperform those augmented using SMOTE and ADASYN. Although CGAN had lower Wasserstein distance scores than SMOTE (Table V), CGAN still performs better in classification than SMOTE, showing that structural patterns in AML data are more important than the distribution of attributes of individual transactions.
- With the standard threshold of 0.5, CGAN-augmented data also yield higher performance than the other datasets, achieving noticeably higher recall and lower FARs.
- Weighted loss (cost sensitive learning) was applied to the GNN models trained with the original datasets to mitigate class imbalance. However, the predictive performance of these models is significantly lower than that of those trained with the augmented datasets. For example, given the PNA model and 90% recall, the original dataset has a FAR of 87.36% vs. 63.79% by CGAN-augmented data.
- Among the classifiers, PNA achieves the lowest FAR overall, likely due to its use of advanced aggregation strategies.

The results from the LI-Medium dataset (Table VII) follow similar trends. GNN models trained on CGAN-augmented data achieve significantly lower FARs compared to those trained on the original datasets as well as SMOTE and ADASYN augmented data. For example, GIN+EU trained with CGAN-augmented data achieves a FAR of 67.64% at 90% recall, whereas the same model trained with ADASYN-augmented data yields a FAR of 80.34%.

Tables VIII and IX present the results for the HI-Small and LI-Small datasets, respectively. In both tables, we observe similar trends as discussed above. For example, the GNN models trained on CGAN-augmented data consistently outperform those trained with SMOTE or ADASYN augmented data and the original datasets. For instance, given the LI-Small dataset and a 90% recall, the PNA model with CGAN-augmented data has a FAR of 71.4%, compared to 82.3% by ADASYN.

The above results confirm that CGAN-generated graph data improve classifier performance more effectively than transaction-level resampled data from SMOTE and ADASYN.

TABLE VI: Recall and FAR (in percentages) of ML detection models on **HI-Medium** dataset

Model	Original Dataset		CGAN		SMOTE		ADASYN	
	Recall	FAR	Recall	FAR	Recall	FAR	Recall	FAR
Target Recall ~90%								
GIN	90.47±0.37	86.12±4.31	90.92±0.83	68.48±3.64	90.65±0.34	78.23±3.13	90.13±0.73	81.75±3.41
GIN+EU	90.58±0.39	85.68±2.51	90.24±0.29	65.79±3.44	90.76±0.58	75.83±3.10	90.09±0.19	80.21±3.22
PNA	90.32±0.46	87.36±3.13	90.84±0.75	63.79±3.41	90.17±0.22	72.97±2.81	90.66±0.48	78.83±3.04
Target Recall ~97%								
GIN	97.23±0.38	95.47±5.61	97.81±0.21	80.21±3.91	97.54±0.25	88.47±3.30	97.16±0.59	91.20±3.43
GIN+EU	97.67±0.46	94.42±5.22	97.45±0.84	77.36±3.54	97.11±0.14	86.38±3.11	97.33±0.97	88.47±3.23
PNA	97.36±0.39	95.63±6.27	97.19±0.93	74.74±3.44	97.48±0.33	82.89±2.83	97.29±0.86	87.64±3.01
Standard Threshold (0.5)								
GIN	39.86±3.61	52.22±5.20	57.40±2.79	31.48±4.11	49.83±2.04	40.66±2.94	45.85±2.18	45.44±3.01
GIN+EU	48.06±6.45	47.44±8.27	66.88±3.90	26.57±5.12	57.27±3.02	36.86±3.77	52.47±3.14	41.67±4.06
PNA	47.42±4.30	30.88±4.26	65.16±3.12	27.14±3.97	56.53±2.39	18.51±2.66	51.79±2.42	23.47±2.81

TABLE VII: Recall and FAR (in percentages) of ML detection models on **LI-Medium** dataset

Model	Original Dataset		CGAN		SMOTE		ADASYN	
	Recall	FAR	Recall	FAR	Recall	FAR	Recall	FAR
Target Recall ~90%								
GIN	90.84±0.15	86.52±5.41	90.33±0.68	69.29±3.19	90.76±0.13	79.21±2.04	90.44±0.47	83.11±2.21
GIN+EU	90.19±0.24	85.80±3.42	90.72±0.66	67.64±3.03	90.38±0.39	77.18±2.05	90.58±0.74	80.34±2.14
PNA	90.25±0.13	87.22±3.97	90.93±0.29	63.96±2.83	90.67±0.64	75.13±2.10	90.14±0.91	79.16±2.10
Target Recall ~97%								
GIN	97.29±0.49	95.77±4.73	97.81±0.89	81.28±3.15	97.42±0.23	90.53±2.12	97.03±0.65	92.44±2.10
GIN+EU	97.68±0.23	95.57±3.92	97.36±0.97	79.69±3.29	97.10±0.76	89.15±2.22	97.24±0.90	90.36±2.11
PNA	97.51±0.32	96.17±4.51	97.45±0.27	76.38±2.93	97.34±0.13	86.52±2.11	97.59±0.48	89.08±2.10
Standard Threshold (0.5)								
GIN	8.07±9.32	94.77±3.60	41.87±2.08	54.00±2.63	33.87±1.52	55.11±1.08	29.82±1.57	59.14±1.11
GIN+EU	5.51±6.82	87.87±2.35	50.43±2.36	41.58±1.72	36.85±1.71	50.22±1.07	33.72±1.76	57.13±1.23
PNA	20.40±0.66	63.50±1.51	51.11±1.32	28.38±1.16	37.83±0.61	49.70±0.94	29.65±0.66	54.34±0.92

C. Training and Inference Time of the CGAN Model

This section reports the training and average inference time of the proposed CGAN model. (The reported inference time is the average time taken to generate a single synthetic graph pattern (e.g., fan-in, fan-out, stack, etc.)) All experiments were conducted on the research computing infrastructure provided by the Digital Research Alliance of Canada, equipped with NVIDIA Tesla V100 GPUs (41). Table X shows the training and average inference time for each of the four datasets listed under the “Augmented” columns in Table II. The results show that CGAN is highly efficient, with the training time ranging from 22.9 seconds to 31.6 seconds, and the average inference time of under one second.

The training and inference time of the downstream GNN classifiers can be found in (1).

VI. CONCLUSION

We propose a CGAN model to generate synthetic data to augment the minority class of extremely imbalanced AML datasets. Unlike SMOTE and ADASYN, which generate isolated samples in flat feature spaces, our method synthesizes full transaction patterns with structural and attribute-level

fidelity. By conditioning on known money laundering (ML) patterns, CGAN captures graph-based dependencies that are characteristic of real-world ML activity. Extensive experiments using four AMLWorld benchmark datasets demonstrate that CGAN-generated graph data improve both detection rates and operational efficiency by significantly reducing false alert rates. The GNN classifiers trained on CGAN-augmented data consistently outperform those trained on data enhanced by traditional oversampling methods like SMOTE and ADASYN.

Beyond AML, our CGAN-based approach offers a generalizable solution for imbalanced datasets in classification tasks involving complex relational structures, such as fraud detection in insurance networks, cyber threat hunting, or molecule property prediction. In future work, we will explore adaptive pattern conditioning, integration with active learning pipelines, and real-time augmentation for evolving AML typologies.

REFERENCES

- [1] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, “Realistic synthetic financial transactions for anti-money laundering models,” in *Proceedings of the 37th Conference on Neural Information*

TABLE VIII: Recall and FAR scores (in percentages) of ML detection models on **HI-Small** dataset

Model	Original Dataset		CGAN		SMOTE		ADASYN	
	Recall	FAR	Recall	FAR	Recall	FAR	Recall	FAR
Target Recall ~90%								
GIN	90.83±0.89	85.96±6.52	90.52±0.36	63.12±8.12	90.45±0.72	70.34±4.62	90.69±0.92	74.62±4.85
GIN+EU	90.66±0.75	85.74±7.02	90.73±0.85	61.90±7.83	90.18±0.23	67.25±4.47	90.50±0.44	71.41±4.76
PNA	90.31±0.66	86.82±7.66	90.92±0.12	58.19±7.11	90.63±0.39	65.81±3.89	90.81±0.78	63.18±3.71
Target Recall ~97%								
GIN	97.33±0.43	93.91±5.81	97.91±0.90	76.09±7.92	97.55±0.78	76.87±4.31	97.18±0.56	87.12±4.76
GIN+EU	97.44±0.48	93.95±6.43	97.86±0.93	72.88±8.19	97.12±0.39	80.92±4.52	97.69±0.42	85.96±4.62
PNA	97.58±0.15	94.21±6.92	97.74±0.36	69.22±7.34	97.11±0.07	79.41±3.87	97.27±0.13	74.90±3.71
Standard Threshold (0.5)								
GIN	38.16±5.92	72.60±7.98	53.78±4.47	62.90±6.21	33.10±3.21	68.45±3.09	30.50±3.33	72.50±3.18
GIN+EU	55.41±5.96	57.71±10.69	66.13±4.68	42.10±6.13	34.11±3.45	50.25±3.26	31.03±3.31	54.50±3.20
PNA	53.15±2.26	41.52±10.67	63.44±2.14	23.79±5.77	35.00±1.91	32.75±1.85	36.41±2.00	30.11±1.78

TABLE IX: Recall and FAR scores (in percentages) of ML detection models on **LI-Small** dataset

Model	Original Dataset		CGAN		SMOTE		ADASYN	
	Recall	FAR	Recall	FAR	Recall	FAR	Recall	FAR
Target Recall ~90%								
GIN	90.72±0.62	89.91±5.27	90.89±0.39	74.21±3.41	90.31±0.82	73.26±1.94	90.68±0.67	83.92±1.81
GIN+EU	90.13±0.46	88.12±5.79	90.41±0.92	70.91±3.14	90.58±0.48	71.47±1.83	90.27±0.66	81.39±1.70
PNA	90.66±0.23	89.36±6.76	90.94±0.62	71.42±2.88	90.22±0.77	74.94±1.63	90.51±0.34	82.31±1.59
Target Recall ~97%								
GIN	97.14±0.13	95.87±5.61	97.38±0.84	82.06±3.41	97.71±0.30	80.35±2.01	97.05±0.58	86.23±1.85
GIN+EU	97.47±0.43	95.82±8.27	97.22±0.20	77.51±3.29	97.83±0.99	79.55±1.91	97.36±0.85	84.83±1.78
PNA	97.51±0.32	95.93±4.81	97.40±0.33	78.20±3.04	97.29±0.70	80.48±1.70	97.62±0.14	85.52±1.62
Standard Threshold (0.5)								
GIN	14.59±2.37	94.86±3.42	24.42±1.94	79.31±2.89	18.95±1.38	78.89±2.12	17.32±1.49	81.10±2.25
GIN+EU	23.26±2.87	78.40±10.83	35.55±2.18	67.10±3.76	29.91±1.51	72.30±2.04	26.23±1.57	75.80±2.16
PNA	16.43±2.62	82.63±5.80	25.66±1.76	74.35±3.01	20.71±1.41	77.80±2.25	19.35±1.48	79.50±2.36

TABLE X: Training time (TT) and average inference time (AIT) in seconds of the proposed CGAN model

Dataset	HI-Small		LI-Small		HI-Medium		LI-Medium	
	TT	AIT	TT	AIT	TT	AIT	TT	AIT
CGAN	24.8	0.56	22.9	0.56	31.6	0.64	29.4	0.59

Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023.

- [2] O. Husain, “13 biggest aml fines (\$500 million plus),” <https://www.enzuzo.com/blog/biggest-aml-fines>, 2024, accessed: 2024-09-05.
- [3] S. Ross and M. Hannan, “Money laundering regulation and risk-based decision-making,” *Journal of Money Laundering Control*, 2007.
- [4] J. Gerlings and I. Constantiou, “Machine learning in transaction monitoring: The prospect of xai,” in *Proceedings of the 56th Hawaii International Conference on System Sciences (HICSS)*, T. X. Bui, Ed., Honolulu, United States, 2023, pp. 3474–3483.
- [5] M. Jullum, A. Løland, R. B. Huseby, G. Ånonsen, and J. Lorentzen, “Detecting money laundering transactions

with machine learning,” *Journal of Money Laundering Control*, vol. 23, no. 1, pp. 173–186, 2020.

- [6] O. Raiter, “Applying supervised machine learning algorithms for fraud detection in anti-money laundering,” *Journal of Modern Issues in Business Research*, vol. 1, no. 1, pp. 14–26, 2021.
- [7] R. I. T. Jensen, J. Ferwerda, K. S. Jørgensen, E. R. Jensen, M. Borg, M. P. Krogh, J. B. Jensen, and A. Iosifidis, “A synthetic data set to benchmark anti-money laundering methods,” *Scientific Data*, vol. 10, no. 1, p. 661, 2023.
- [8] B. Oztas, D. Cetinkaya, F. Adedoyin, M. Budka, H. Dogan, and G. Aksu, “Enhancing anti-money laundering: Development of a synthetic transaction monitoring dataset,” in *2023 IEEE International Conference on e-Business Engineering (ICEBE)*. IEEE, 2023, pp. 47–54.
- [9] E. Altman, “Ibm transactions for anti-money laundering (aml),” 2019, accessed: 2025-06-06.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [11] H. He, Y. Bai, E. A. Garcia, and S. Li, “Adasyn: Adaptive

- synthetic sampling approach for imbalanced learning,” in *IEEE International Joint Conference on Neural Networks (IJCNN)*, 2008, pp. 1322–1328.
- [12] B. N. Pambudi, I. Hidayah, and S. Fauziati, “Improving money laundering detection using optimized support vector machine,” in *International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*. IEEE, 2023.
- [13] Y. Zhang and P. Trubey, “Machine learning and sampling scheme: An empirical study of money laundering detection,” *Computational Economics*, vol. 54, pp. 1043–1063, 2019.
- [14] J. Schmidt, D. Pasadakis, M. Sathe, and O. Schenk, “Gamlnet: A graph-based framework for the detection of money laundering,” Preprint, 2025.
- [15] P. Tertychnyi, T. Lindström, C. Liu, and M. Dumas, “Detecting group behavior for anti-money laundering with incomplete network information,” in *IEEE International Conference on Big Data (Big Data)*. IEEE, 2022.
- [16] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” arXiv preprint arXiv:1411.1784, 2014.
- [17] K. Dolan, “Aml trends in 2022: What you need to know,” <https://www.idc.com/getdoc.jsp?containerId=US48661022>, IDC, Tech. Rep., 2022, accessed: 2023-01-10.
- [18] T. Suzumura and H. Kanezashi, “Anti-money laundering datasets,” <https://github.com/IBM/AMLSim>, 2021.
- [19] M. Weber, J. Chen, T. Suzumura, A. Pareja, T. Ma, H. Kanezashi, T. Kaler, C. E. Leiserson, and T. B. Schardl, “Scalable graph learning for anti-money laundering: A first look,” 2018.
- [20] P. Branco, L. Torgo, and R. P. Ribeiro, “A survey of predictive modeling on imbalanced domains,” *ACM Computing Surveys (CSUR)*, vol. 49, no. 2, pp. 1–50, 2016.
- [21] H. Wang and H. He, “Survey on deep learning with class imbalance,” *Journal of Big Data*, vol. 6, no. 1, pp. 1–54, 2019.
- [22] A. Gupta and R. Sharma, “A broad review on class imbalance learning techniques,” *Applied Soft Computing*, vol. 135, p. 110026, 2023.
- [23] S. Ghosh, S. K. Kamila, S. Maity, and D. De, “Ensemble learning for class imbalance: a review,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 12, no. 2, p. e1441, 2022.
- [24] N. Bakhshinejad, “A graph-based deep learning model for anti-money laundering,” Master’s thesis, York University, Toronto, Ontario, April 2023.
- [25] B. Stein *et al.*, “Network analytics for anti-money laundering,” arXiv preprint arXiv:2405.19383, 2024.
- [26] W. Chen, K. Yang, Z. Yu, Y. Shi, and C. L. P. Chen, “A survey on imbalanced learning: latest research, applications and future directions,” *Artificial Intelligence Review*, 2024.
- [27] J. L. Leevy, T. M. Khoshgoftaar, R. A. Bauder, and N. Seliya, “A survey on addressing high-class imbalance in big data,” *Journal of Big Data*, vol. 5, no. 1, p. 42, 2018.
- [28] G. Chandrashekar and F. Sahin, “A survey on feature selection methods,” *Computers & Electrical Engineering*, vol. 40, no. 1, pp. 16–28, 2014, publisher: Elsevier.
- [29] K. Sintayehu and H. Seid, “Developing anti-money laundering identification using machine learning techniques,” *Irish Interdisciplinary Journal of Science Research (IIJSR)*, vol. 7, no. 1, pp. 64–74, 2023.
- [30] M. Caglayan and S. Bahtiyar, “Money laundering detection with node2vec,” *Gazi University Journal of Science*, vol. 35, no. 3, pp. 854–873, 2022.
- [31] H. S. Assumpcao, F. Souza, L. L. Campos, V. T. de Castro Pires, P. M. L. de Almeida, and F. Murai, “Delator: Money laundering detection via multi-task learning on large transaction graphs,” in *IEEE International Conference on Big Data (Big Data)*. IEEE, 2022.
- [32] N. Bakhshinejad, U. T. Nguyen, S. Ghahremani, and R. Soltani, “A graph-based deep learning model for the anti-money laundering task of transaction monitoring,” in *16th International Joint Conference on Computational Intelligence (IJCCI)*. SCITEPRESS – Science and Technology Publications, Lda, 2024, pp. 496–507.
- [33] X. Luo, X. Han, W. Zuo, Z. Xu, Z. Wang, and X. Wu, “A dynamic transaction pattern aggregation neural network for money laundering detection,” in *IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2022.
- [34] A. Băltoiu, A. Pătraşcu, and P. Irofti, “Community-level anomaly detection for anti-money laundering,” 2019.
- [35] A. El-Kilany, A. M. Ayoub, and H. M. E. Kadi, “Detecting suspicious customers in money laundering activities using weighted hits algorithm,” in *5th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*. IEEE, 2024.
- [36] C. Villani, *Optimal Transport: Old and New*, ser. Grundlehren der mathematischen Wissenschaften. Springer, 2009, vol. 338.
- [37] F. J. M. Jr, “The kolmogorov-smirnov test for goodness of fit,” *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.
- [38] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *International Conference on Learning Representations (ICLR)*, 2019.
- [39] W. Hu, M. Fey, H. Ren, M. Nakata, and J. Leskovec, “Open graph benchmark: Datasets for machine learning on graphs,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 22 118–22 133, 2020.
- [40] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Velickovic, “Principal neighbourhood aggregation for graph nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [41] D. R. A. of Canada, “Digital research infrastructure,” <https://alliancecan.ca/>, 2025.